Python Mid-Sem Revision Notes

Dictionaries • Sets • Input & Output • Built-in Functions • OOP • Exceptions

Concept-first revision with simple code examples, important methods, differences and exam-oriented questions.

1. Python Dictionaries

A dictionary is a mutable collection that stores data as **key-value pairs**. Keys are unique and are used to access values.

Creating and Accessing

```
student = {
    "name": "Rahul",
    "age": 20,
    "marks": 85
}

print(student["name"])
print(student["marks"])
```

Adding and Updating

```
student = {"name": "Rahul", "marks": 85}

student["age"] = 20       # add
student["marks"] = 90     # update

print(student)
```

get()

```
student = {"name": "Rahul", "marks": 85}

print(student.get("name"))
print(student.get("age"))
print(student.get("age", "Not Found"))
```

Using get() avoids a KeyError when the requested key is missing.

Important Dictionary Methods

Method	Purpose
keys()	Returns all keys.
values()	Returns all values.
items()	Returns key-value pairs.
get(key)	Returns value for a key.
update(dict)	Adds/updates entries.
pop(key)	Removes the specified key and returns its value.
popitem()	Removes the last inserted key-value pair.
clear()	Removes all entries.

Traversing a Dictionary

```
student = {"name": "Rahul", "marks": 85}

for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, ":", value)
```

Dictionary Frequency Example

```
words = ["python", "java", "python", "c", "java", "python"]
frequency = {}

for word in words:
    if word in frequency:
        frequency[word] += 1
    else:
        frequency[word] = 1
```

```
print(frequency)
```

Important Points

Dictionary keys must be unique. If the same key is assigned again, its old value is replaced. Dictionaries are mutable and are useful for fast key-based lookup and frequency counting.

2. Python Sets

A set is an unordered collection of unique elements. It is mutable, but its individual elements must be hashable.

Creating a Set

```
numbers = {10, 20, 30, 20}
print(numbers)     # duplicates removed

empty_set = set()  # {} creates an empty dictionary
```

Adding and Removing

```
s = {10, 20, 30}

s.add(40)
s.remove(20)
s.discard(50)

print(s)
```

remove(x) raises KeyError if x is absent. **discard(x)** does nothing if x is absent.

Set Operations

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print(A | B)   # union
print(A & B)   # intersection
print(A - B)   # difference
print(A ^ B)   # symmetric difference
```

Important Set Methods

Operation / Method	Meaning
union() / |	All elements from both sets.
intersection() / &	Common elements.
difference() / -	Elements in first set but not second.
symmetric_difference() / ^	Elements in either set but not both.
add(x)	Adds one element.
remove(x)	Removes x; error if absent.
discard(x)	Removes x; no error if absent.
pop()	Removes an arbitrary element.
clear()	Removes all elements.

Removing Duplicates from a List

```
numbers = [10, 20, 10, 30, 20, 40]
unique = set(numbers)
print(unique)
```

If the original order must be preserved, a set alone should not be used as the final ordered representation.

3. Input and Output in Python

input()

input() reads user input and returns it as a string.

```
name = input("Enter name: ")
age = int(input("Enter age: "))
marks = float(input("Enter marks: "))
```

print()

```
name = "Rahul"
marks = 90

print("Name:", name)
print("Marks:", marks)
print("Name =", name, "Marks =", marks)
```

Formatted Output using f-strings

```
name = "Rahul"
marks = 90

print(f"Name: {name}")
print(f"Marks: {marks}")
print(f"{name} scored {marks} marks")
```

Useful print() Arguments

```
print("A", "B", "C", sep="-")
print("Hello", end=" ")
print("World")
```

sep controls the separator between values. end controls what is printed after the output; by default it is a newline.

4. Python Built-in Functions

Built-in functions are functions provided by Python and available without importing a library.

High-Priority Built-ins

Function	Use
len(x)	Number of elements/characters.
sum(x)	Sum of numeric elements.
max(x)	Largest value.
min(x)	Smallest value.
sorted(x)	Returns a new sorted list.
type(x)	Returns the type of x.
range()	Generates a sequence of integers.
abs(x)	Absolute value.
round(x)	Rounds a number.
int(), float(), str()	Type conversion.

Example

```
marks = [80, 35, 90, 60, 75]

print("Length:", len(marks))
print("Total:", sum(marks))
print("Maximum:", max(marks))
print("Minimum:", min(marks))
print("Sorted:", sorted(marks))
print("Type:", type(marks))
print("Absolute:", abs(-10))
print("Rounded:", round(12.567, 2))
```

Common Exam Difference: sort() vs sorted()

sort() is a list method and changes the original list. **sorted()** is a built-in function and returns a new sorted result.

```
a = [30, 10, 20]
a.sort()
print(a)

b = [30, 10, 20]
c = sorted(b)
print(c)
print(b)
```

5. Python Object-Oriented Programming

OOP organizes programs using classes and objects. The main concepts for your syllabus are class, object, constructor, self, methods, encapsulation, inheritance, overriding and overloading-like behavior.

Class and Object

```
class Student:
    def display(self):
        print("Student details")

s1 = Student()
s1.display()
```

Class is a blueprint. **Object** is an instance of a class.

Constructor and self

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name)
        print(self.marks)

s1 = Student("Rahul", 90)
s1.display()
```

__init__() is automatically called when an object is created. **self** refers to the current object.

Encapsulation

Encapsulation means combining data and methods inside a class. It helps organize and control access to object data.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name, self.marks)
```

Inheritance

Inheritance allows a child class to acquire properties and methods from a parent class.

```
class Person:
    def __init__(self, name):
        self.name = name

    def display(self):
        print("Name:", self.name)

class Student(Person):
    def __init__(self, name, reg_no):
        super().__init__(name)
        self.reg_no = reg_no

s = Student("Rahul", "23CSE001")
s.display()
print("Register:", s.reg_no)
```

Types of Inheritance

Type	Structure
Single	A → B
Multiple	A + B → C
Multilevel	A → B → C
Hierarchical	A → B and A → C
Hybrid	Combination of two or more inheritance types.

Method Overriding

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

super()

```
class Animal:
    def sound(self):
        print("Animal sound")

class Dog(Animal):
    def sound(self):
        super().sound()
        print("Dog barks")

d = Dog()
d.sound()
```

Method Overloading in Python

Python does not support traditional method overloading by defining multiple methods with the same name. Default arguments or *args can provide overloading-like behavior.

```
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

c = Calculator()
print(c.add(10, 20))
print(c.add(10, 20, 30))
```

Overloading vs Overriding

Overloading	Overriding
Different argument requirements	Child changes inherited implementation.
Usually within one class	Requires parent-child relationship.
Python uses default arguments/*args for similar behavior	Supported directly through inheritance.

6. Exception Handling

An exception is an error that occurs during program execution. Exception handling prevents the program from terminating unexpectedly and allows an appropriate response.

Syntax Error vs Exception

Syntax error: Python cannot understand the program syntax. **Exception:** the syntax is valid, but an error occurs while executing the program.

try-except

```
try:
    a = int(input("Enter a: "))
    b = int(input("Enter b: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Enter valid numbers")
```

else and finally

```
try:
    n = int(input("Enter number: "))
    print(100 / n)
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Operation successful")
finally:
    print("Program finished")
```

try: contains risky code. **except:** handles an exception. **else:** runs when no exception occurs. **finally:** runs whether an exception occurs or not.

Common Exceptions

Exception	Typical cause
ValueError	Invalid value, e.g. int('abc').
ZeroDivisionError	Division by zero.
IndexError	Invalid list/sequence index.
KeyError	Missing dictionary key.
TypeError	Operation on incompatible types.
NameError	Using an undefined variable.

Handling List IndexError

```
numbers = [10, 20, 30]

try:
    i = int(input("Enter index: "))
    print(numbers[i])
except IndexError:
    print("Invalid index")
except ValueError:
    print("Enter an integer")
```

Handling Dictionary KeyError

```
student = {"name": "Rahul", "marks": 90}

try:
    key = input("Enter key: ")
    print(student[key])
except KeyError:
    print("Key does not exist")
```

raise

raise is used to deliberately generate an exception when a condition is invalid.

```
age = -2

if age < 0:
    raise ValueError("Age cannot be negative")
```

7. Final Exam Revision

Must know: dictionary key-value pairs and methods; set operations and duplicate removal; input/output and f-strings; built-in functions; class/object/__init__/self; five inheritance types; overriding, super() and overloading-like behavior; try-except-else-finally and common exceptions.

High-Value Conceptual Questions

1. Define a dictionary. Explain keys, values and important dictionary methods.
2. Differentiate dictionary and set.
3. Explain union, intersection and difference of sets.
4. Differentiate remove() and discard() in sets.
5. Explain input() and print() with examples.
6. Explain f-string formatting.
7. Explain any five Python built-in functions with examples.
8. Differentiate sort() and sorted().
9. Define class and object with an example.
10. Explain __init__() and self.
11. Explain encapsulation.
12. Explain inheritance and its five types.
13. Explain method overriding with a program.
14. Explain the use of super().
15. Does Python support traditional method overloading? Explain.
16. Differentiate method overloading and method overriding.
17. Define exception and explain exception handling.
18. Explain try, except, else and finally.
19. Explain ValueError, ZeroDivisionError, IndexError, KeyError and TypeError.
20. Write a program using multiple except blocks.

Revision strategy: For each concept remember: definition → syntax/method → one small code example → one difference or exam question. For programming questions, practice writing without looking at the solution.